附录A. JavaScript快速入门

在如今琳琅满目的编程语言中，JavaScript是一个非常特殊的存在。如果从语言设计的角度来分析，JavaScript应该被归类为基于原型的、解释型的高级编程语言。这也就是说，尽管JavaScript在语法上与Java、C/C++非常相似，但在程序设计的思路上，它受Self和Scheme这类语言的影响恐怕会更多一些，因此在具体使用方式上可能更接近于后者。除此之外，JavaScript还是一门支持多种编程范式的语言，它支持面向对象编程、指令式编程以及函数式编程，因而具有极为灵活的表达能力。而在适用领域方面，如今的JavaScript也从最初的纯浏览器端脚本语言，逐步发展成了可在浏览器端、移动设备端、桌面应用端以及服务器端通用的强大编程语言。

正是由于这门语言具有如此的使用深度和广度，以及在语法、设计理念上的复杂度，使得笔者认为在具体使用该语言进行面向实际的项目实践之前，读者需要先花时间了解一下它的基本语法以及这些语法背后所反映的设计理念。

A.1 语言概况简介

曾经在相当长的一段时间里，人们使用网页浏览器所能访问到都只不过是一组依靠超链接简单串联在一起的HTML文档而已。这些文档既无数据处理能力，也无法响应用户的操作，简单到甚至都不能被称之为“程序”，充其量不过是一本被放在互联网上供人浏览的“书”罢了。大概正是因为如此，如今用来阅读这些HTML文档的这个工具才会被叫做“浏览器”吧。然而，随着互联网上的业务需求与日俱增（例如电子商务、线上交友、视频分享等），开发者们越来越希望自己所创建的网站能具有更强大的数据交互功能，并能即时响应用户的操作。于是，JavaScript就应运而生了。

A.1.1 ES标准规范

JavaScript的历史开始于1995年，当时世界上最成功的网页浏览器提供商 —— 网景公司聘请了一个名叫布兰登·艾克（Brendan Elch）的人，希望他研发一个能与Java语言搭配使用，语法上也与其相似的浏览器端编程语言。后者也没有辜负网景公司的重托，仅用了十天就完成了该语言的原型设计，并在Netscape Navigator 2.0的Beta版中发布了它，当时这门语言的名称还是LiveScript。等到同年的12月，网景公司在Netscape Navigator 2.0 Beta 3版发布时又将它重新名为JavaScript，相传这样做的目的主要是想让这门新生的编程语言蹭一下Java的“热度”，相信他们当时可能也没有想到，这个名字反而在日后成为了大众对该语言的诸多误解之一，因为在事实上，Java和JavaScript之间就像印度和印度尼西亚一样，完全没有任何从属关系。

然后，当时间来到了1996年11月，为了反制微软公司一路以来在网页浏览器领域所采取的恶性竞争策略，同时也是为了规范JavaScript语言的实现，网景公司正式向ECMA（欧洲计算机制造商协会）提交了针对该语言的标准化草案。接着就在次年的6月，ECMA就以当下的JavaScript语言实现为基础制定了ECMA-262标准规范。自此之后，JavaScript核心部分的实现也被称为`ECMAScript [标准版本号]`，我们常将其简写为`ES [标准版本号]`。目前市场上所使用的ECMAScript基本以ES5、ES6为主，本书的内容也将会主要会以ES5/ES6为标准来展开。这样做既有助于读者理解JavaScript发展至今所形成的设计理念与编程思想，也有助于这门语言的初学者们快速上手并实验本书中所呈现的示例，而不必因为相关执行环境对标准的兼容度而发愁。

A.1.2 组成与特性

正如之前所说，JavaScript最初只是一门依附于网页浏览器的脚本语言，但随着Node.js运行环境的出现，它如今已经发展成了横跨Web应用前后端的全能型编程语言。所以，当我们讨论JavaScript这个语言名称的时候，必须要了解该语言除了核心部分的ECMAScript标准之外，还包含了其所在的宿主环境。譬如，当我们讨论基于网页浏览器的JavaScript的时候，就应该知道这时候的讨论内容除了ECMAScript标准所规定的语法和基本对象之外，通常还会涉及到用于处理网页内容的文档对象模型（简称DOM），和用于处理浏览器事务的浏览器对象模

型（简称BOM）。但到了Node.js运行环境中，DOM和BOM就不存在了，这时候，我们需要关心的就是Node.js所提供的核心模块，以及各种特定用途的第三方模块了。

总而言之，JavaScript这个名称所代表的已经不只是ECMAScript标准所规范的一门脚本语言了，它还会涉及到这门语言所在的宿主环境与应用框架，在之后学习JavaScript的过程中，我们会越来越意识到这一点的重要性，这种意识将有助于大家理解JavaScript在Web应用的前后端开发中扮演的不同角色，而不至于会产生混淆。

当然，我们在这里所说的“全能型编程语言”仅仅指的是JavaScript适用的领域很广，并非是认为可以用这门语言来解决所有的编程问题。JavaScript自诞生以来一以贯之的设计理念让这门语言具备了一些与众不同的特性，这些特性基本上决定了它的编程思想以及专长的领域。下面，我们先来初步罗列一下这些特性。

- **动态化类型**：和大多数脚本语言一样，JavaScript中的数据类型是直接取决于变量中的“值”的，变量本身没有数据类型上的约束，这是它们与编译型语言最大的区别之一。也就是说，JavaScript中的同一个变量可以存储不同类型的值，例如，如果我们在JavaScript的代码中定义了一个名为`x`的变量，`x`的值是可以初始值为数字，然后在执行过程中被重新赋值为字符串的，JavaScript的宿主环境会负责自动识别变量的类型。
- **多范式编程**：JavaScript虽然在语法上与Java、C/C++非常类似，但在内在的设计哲学上，它则更接近于Self和Scheme这一类语言。也就是说，它既支持面向对象编程，也支持指令式编程和函数式编程，因而具有极为灵活的表达能力。
- **单线程执行**：由于最初脱胎于网页浏览器的关系，JavaScript社区一直习惯采用单一线程的执行模式（尽管如今有了支持多线程的Worker组件），这一习惯即使到了Node.js运行环境中也基本如此。采用单一线程的最大好处是不用像多线程编程那样处理很容易产生bug的同步问题，这就从根本上杜绝了死锁问题，也避免了线程上下文交换所带来的性能上的开销。当然了，单一线程的执行方式也有它自身的弱点，譬如，它无法充分发挥多核处理器的性能、一个错误就会导致整个程序崩溃，以及执行大量计算时会因长期占用处理器而影响其他异步I/O的执行。
- **事件驱动**：在Web应用开发中，JavaScript之所以能在浏览器这一端扮演越来越重要的角色，很大程度上就是得益于其具有与桌面应用相似的事件驱动模型。当然，这种编程模型虽然具有轻量级、松耦合等优势，但在多个异步任务的场景下，由于程序中的各个事件是彼此独立的，它们之间的协作就成为了一个需要我们费心解决的问题。
- **异步编程**：在目前流行的Vue.js等前端框架以及Node.js运行环境提供的接口中，我们都可以很容易地观察到，其大部分操作都是以异步调用的方式来进行的，而这些异步调用往往都会以回调函数的形式存在，这已经成为了使用JavaScript编程的一大特色。当然，虽然大家都认为回调函数是执行异步调用并接收其返回数据的最佳方式，但这种方式也会导致代码的编写顺序与其具体执行顺序的不一致，对于很多习惯同步思路编程的人来说，阅读这样的代码会是一个不小的挑战。另外在流程控制方面，也会由于程序中穿插了各种异步方法和回调函数，让代码在可读性上也远没有常规的同步方式那么一目了然，这也会给程序的调试和维护工作带来一定的麻烦。

A.1.3 适用领域

正如之前所说，如果想判断一门编程语言是否适用于某个领域，很大程度就要去分析该领域是否能发挥出语言的特性优势。既然我们已经对JavaScript的语言特性有了一定的了解，接下来就可以对这门语言的适用领域做一些分析了。这些分析将有助于初学者们明确JavaScript适合用来解决什么问题，不适合用来解决什么问题，以便厘清自己的学习需求和努力方向。现在，让我们先来罗列一下适合用JavaScript来解决的问题领域：

- **浏览器端的应用**：JavaScript在浏览器端的优势是最显而易见，且无可争议的，毕竟这门语言最初就是为解决这一领域的问题而设计的。正是由于JavaScript赋予了网页在浏览器端强大的用户交互能力，我们才迎来了电子商务、云端办公、社交网络等各路Web应用蓬勃发展的Web 2.0时代。

- **轻量级的服务器应用**：Node.js运行环境的出现让JavaScript的适用领域得以扩展到了网页浏览器之外，其中很大的一块就是服务器端的应用。与ASP、PHP这些传统的服务器端脚本相比，JavaScript支持事件驱动、异步编程的特性使它在实现轻量级数据密集型的服务器应用方面享有了一些高性能、高负载的优势。当然在另一方面，单线程执行和非阻塞I/O的特性也让该语言的能力在资源利用率和安全性方面受到了一些限制，使其不适合被用来实现需要大规模并行计算，或对数据安全有相当要求的应用。
- **轻量级的桌面应用**：Electron框架的出现也让我们可以用JavaScript来实现一些适用于事件驱动、异步编程、非阻塞型I/O等特性的轻量级桌面应用，譬如目前流行的VS Code、Atom等代码编辑器都是基于这一框架的JavaScript应用。但同样是由于其单线程执行的特性使JavaScript无法充分利用多核处理器的计算资源，因而也不适合用来实现需要大规模并行计算的桌面应用。
- **富媒体式的应用**：在HTML 5出现之前，市面上主要用于创作富媒体的应用程序（如Flash）大多采用的都是ActionScript脚本，由于后者也是一种基于ECMAScript标准的脚本语言，所以，也可被视为是JavaScript的一种应用。当然，目前这种形式的应用正在逐渐被人遗忘，毕竟HTML 5为我们提供了更好的选择。

除此之外，JavaScript有时候还会被用来实现一些网页浏览器的扩展与插件、移动端的一些应用，甚至是一些用于系统管理的命令行脚本。总而言之，我们希望再三强调的是，虽然目前JavaScript已经发展成了一门无处不在的全能型编程语言，但还是得注意语言特性的发挥，能发挥出特性的优势才是它真正适用的领域，反之，不分场合地强行使用这门语言可能只会让我们弄巧成拙、事倍功半。

A.2 基本语法学习

自《The C Programming Language》这本程序设计领域的经典教程问世以来，在命令行终端中输出“Hello World”字样都已经约定俗成式地成为了学习一门新的编程语言或者测试该语言执行/调试环境的第一个演示程序。这样做的好处是先让大家对要学习的语言，以及如何执行该语言的程序有一个整体的印象。这既可以确认语言的执行/调试环境是否搭建完成，也可以为接下来关于语法元素的介绍做一个开场或者说提供一个切入点。所以，接下来就闲话少说，让我们来编写JavaScript版的“Hello World”吧！其具体创建步骤如下。

1. 在之前创建的`Examples/02_studynodejs`目录下创建一个名为`sayHello`的目录，然后在该目录下创建一个`sayHelloInConsole.js`的脚本文件，并其中输入如下代码。

```
// 第一个JavaScript脚本
// 作者：owlman

const name = "owlman";
console.log("Hello ", name);
```

2. 在保存文件并退出编辑器之后，打开命令行终端进入到`sayHello`目录下，并执行`node sayHelloInConsole.js`命令，如果之前搭建的执行/调试环境一切正常，我们就会看到程序输出结果如图A-1所示。

```
Examples\02_studynodejs\sayHello on master [?] via v20.5.0
❯ node .\sayHelloInConsole.js
Hello  owlman
```

图A-1 第一个JavaScript脚本

A.2.1 为代码编写注释

在上面这段“Hello World”程序中，我们首先看到的是三行以`//`开头的注释信息。在JavaScript中，注释主要有以下两种形式。

- `// [注释内容]`**形式**：这种形式的注释可以在代码的任意地方以`//`开头来编写注释信息，直至其所在行结束为止，例如像下面这样。

  ```
  //JavaScript支持加减法运算
  const a = 100;
  const b = 50;
  const c  = a + b; // 请问 c = ?
  ```

- `/* [注释内容] */`**形式**：这种形式的注释可以在代码的任意地方以`/*`开头编写注释信息，然后以`*/`结束。由于这种形式的注释内容可以包含换行符，因此通常用于编写多行注释，譬如对于之前这段“Hello World”程序中的那三行注释，我们还可以有下面这种写法。

  ```
  /*
  * 第一个JavaScript脚本
  * 作者: owlman
  */

  const name = "owlman";
  console.log("Hello ", name);
  ```

和所有的编程语言一样，注释也不会被JavaScript解释器视为可执行代码，它主要用于说明相关代码的作用，以此来提高代码的可读性，方便日后的维护工作，譬如在上面这段程序中，我们用注释说明了这个程序的基本信息。当然，我们永远应该记得：JavaScript本身就是一门语言，它的作用除了让机器按照它的意图正确执行之外，也应该让使用这门语言的人类看得懂它的意图，注释只是辅助说明，可不是充当翻译。换而言之，我们应该尽量用代码本身来表达它所要表达的意图，而不是处处都借助注释，譬如下面代码中的注释很大程度上是画蛇添足的：

```
const a = 100;    // 变量 a = 100
const b = 50;      // 变量 b = 50
const c = a + b;  // 变量 c = a + b
```

A.2.2 变量与操作符

下面，让我们继续对“Hello World”程序的讲解。在看完程序基本信息的注释说明之后，紧接着看到的是`const name = 'owlman"`这行代码，它的作用是在程序中定义一个变量。变量这个概念最早源自于数学中的代数运算，为了方便演算过程的书写，我们通常会用一些简单的字母来指代演算过程中不断变化的已知量或未知量，毕竟在公式中写`x`、`y`、`z`这样的字母总是要比写10^{2019}或者`3.141592653589`这样的数字简单方便多了。

到了计算机程序中，变量的概念得到了进一步扩展，除了是某个数据值的指代外，它还关联着计算机中用于存储该数据的一块内存空间，换而言之，变量现在还是程序用来存储某个数据的容器。当然了，这些容器既然能被称为“变”量，也就说明它们所存储的数据是会随着程序的执行而变化的。由于变量是程序所要操作的基本对

象，所以在编写程序时，定义变量往往是我们首先要做的工作。下面，我们就来看看具体该如何定义并使用变量吧！

A.2.2.1 变量的定义

在JavaScript中，定义变量的动作是用定义语句来描述的（关于语句的概念，我们稍后会做详细介绍），它主要由以下三部分组成。

- **变量的定义指令**：在ES6标准发布之前，我们一直是使用`var`关键字来定义变量的，而ES6则另外增加了`let`、`const`这两个新的关键字，以便分别用于定义可改变量与不可改变量。
- **变量的名称**： 在JavaScript中，每个变量都必须要有相应的变量名，这些变量名通常应由一个或多个字母、数字、下划线和美元符号`$`组成，并且只能以字母、下划线和`$`符号开头。另外，变量名也不能与JavaScript语言自身内部使用的保留字相同。关于变量的命名规范，我们稍后会详细说明。
- **变量的初始值**： 这部分的内容是可选的，但为了避免程序出现各种无意义的操作，我们通常都会在定义变量时赋予它一个初始值。

接下来，让我们来具体讨论一下变量的命名问题。根据ECMAScript标准，变量名可以由字母、数字、下划线及美元符号`$`以任何顺序排列组合而成，并且只能以字母、下划线和`$`符号开头，像下面这些变量名都是不被允许的。

```
const 2day;
const "Week;
const \Month;
const /Year;
```

另外需要记住的是，JavaScript中的变量名是大小写敏感的，例如`name`和`Name`在JavaScript代码中是两个完全不同的变量。除了这些命名规则之外，每一种编程语言都会有一些特殊用途的保留字，这些保留字不能被用作变量名。下面，让我们先来罗列一下语言本身的保留字。

保留字	保留字	保留字	保留字	保留字
abstract	arguments	boolean	break	byte
Case	catch	char	class	const
continue	debugger	default	delete	do
double	else	enum	eval	export
extends	false	final	finally	float
For	function	goto	if	implements
import	in	instanceof	int	interface
Let	long	native	new	null
package	private	protected	public	return
Short	static	super	switch	synchronized
This	throw	throws	transient	true

保留字	保留字	保留字	保留字	保留字
Try	typeof	var	void	volatile
While	with	yield		

表A-1 JavaScript中的保留字

与此同时，我们在原则上还应该避免使用一些JavaScript标准库中使用的对象及其属性和方法的名称。下面，我们继续来罗列一下使用时需要注意，不要意外与标准库冲突的变量名：

变量名	变量名	变量名	变量名	变量名
Array	Date	eval	function	hasOwnProperty
Infinity	isFinite	isNaN	isPrototypeOf	length
Math	NaN	name	Number	Object
prototype	String	toString	undefined	valueOf

表A-2 会与标准库发生冲突的变量名

最后，出于代码可读性方面的考虑，变量的命名应该尽量使用有意义的单词或单词组合，不能太过随意。在这里，笔者个人更倾向于建议读者应在变量的命名上遵守某种一致的命名规范。譬如匈牙利命名法，这套命名规范建议我们将变量的数据类型也写到变量名中，例如在`const strname="owlman"`这个变量定义中，我们用`str`表明了这是个字符串类型的变量。再譬如驼峰命名法，遵守这种命名规范的变量名通常由一个以上的单词组成，除了首个单词的字母不是大写，其余所有单词的首字母均为大写，例如`myName`、`myBook`、`someValue`、`getObject`等。

A.2.2.2 变量的类型

在计算机中，如果程序想要对某一块内存空间中数据进行存储和操作，首先要明确的是该空间内数据的存储方式和操作方式。譬如，变量中存储的是数据本身还是数据在内存中的位置？这将决定着这些数据的复制方式。再譬如，变量中的数据可以执行什么操作？是算术运算还是逻辑判断，亦或是文本处理？这就需要我们对这些内存空间中的数据，即变量的值进行归类，譬如用来做算术运算的数据归为一类，用于文本处理的数据则归为另一类，在编程术语上，这些归类被人们约定俗成地称之为“类型（type）”。简而言之，就是变量中的值所属的类型决定了该变量的存储形式及其可以执行的操作。下面，我们从变量值的数据类型开始介绍。

数据类型

具体到JavaScript语言中，我们可以将变量的数据类型分成基本数据类型和对象类型两种。关于对象类型，我们将会在稍后会单独介绍，现在先来看看这门语言所支持的五种基本类型。

- **Number**：即数字类型，这一类型的数据包括正负整数与浮点数（例如：`0`、`1`、`-1`、`-0.25`、`3.14`等）、八进制数与十六进制数（例如：`0377`、`0xff`等）、指数（例如：`1e+2`、`10e+3`等）以及`Infinity`与`NaN`这两个特殊数字。其中，`Infinity`相当于JavaScript世界中的“无穷大”，和数学中有正无穷大和负无穷大一样，这类值也是有`Infinity`和`-Infinity`两种的。而`NaN`则表示的是一种不符合规范但仍属于数字类型的数字，例如当我们让一个数字与一个字符串相乘时，结果就会是一个`NaN`。

- **String**：即字符串类型，这一类型的数据主要指的是被反引号、单引号或双引号所包括起来的、由任意数量字符组成的字符序列，例如：`'a'`、`'123'`、`"one"`、`"one two three"`等。另外，出于代码可读性方面的考虑，笔者个人更倾向于建议读者应在字符串的表示风格上保持一致，不要时而用单引号，时而用双引号。当然，ES6新增的反引号表示的字符串形式比较特殊，它主要用于建立模版字符串。例如，当我们需要将某个变量的值嵌入到某个 HTML 标签中，就可以像下面这样做。

  ```
  const name = "owlman";
  console.log(`<h1> ${ name } </h1>`); // 输出: <h1> owlman </h1>
  ```

  在模版字符串中，我们只需要使用`${ 变量名 }`这样的语法就可以将现有变量的值嵌入到字符串中了，这里的`变量名`可以是任何已在别处定义的变量的名称。

- **Boolean**：即布尔类型，这一类型的数据只有`true`和`false`两种值，主要用于关系运算和逻辑运算。关于这两种运算，我们稍后会详细说明。

- **`undefined`与`null`**： 这是JavaScript中的两个特殊值，当我们访问一个不存在的变量，或未被初始化的变量时，程序就会得到一个`undefined`值。而`null`着通常是指没有值、空值，不代表任何东西。`null`与`undefined`最大的不同在于，被赋予`null`的变量会被认为是被初始化了的，只不过它是个空值。关于空值的概念，我们以后会通过一些具体的示例来解释。

在JavaScript中， 任何不属于上述五种基本类型的值都会被视为对象，某些宿主环境甚至将`null`值也视为一个对象。我们在之后的内容中会深入阐述对象的概念，现在还需要再进一步来探讨一下类型与值的关系。

正如我们在之前所说的，作为一种动态类型的脚本语言，JavaScript 的数据类型是直接与“值”，而不是与变量相关联的。这也就意味着，在JavaScript代码的执行过程中，变量的类型是可以随着变量的值而变化的，例如，如果我们在代码中定义了一个名为`x`的变量，`x`的值是可以初始值为数字，然后在执行过程中被重新赋予其他类型的值的，JavaScript 的宿主环境会负责自动识别变量的类型。

存储类型

在上一节中，我们讨论的是变量“值”所属的数据类型，它决定的是这些值可执行的操作。但正如我们所说，在JavaScript中，变量只负责存储数据，与数据类型并没有直接的关联，这是它与Java、C/C++这些强类型语言的一个重大区别。那么，变量在存储上有没有分类型呢？答案是有的，变量的存储类型决定了变量值的存储方式。在这一节中，我们就来讨论一下变量的存储类型。

根据ECMAScript标准的规定，变量按照其存储方式可分为以下两种类型：

- **原始类型**：在这一类型存储方式下，变量的值往往会直接存储在变量所在栈空间中，由于这种方式所占的内存空间是固定的，所以通常用来存储一些简单且需要快速存取的数据。
- **引用类型**： 在这一类型存储方式下，变量的值往往会被分配在堆空间中，然后将其在堆空间中的位置存储在变量中，这通常用来存储一些复杂且内存开销较大的数据。

那么，当我们为一个变量赋值时，该如何判断该值的存储类型呢？答案很简单，在JavaScript中，只有基本类型是以原始值的方式存储的。也就是说，除了`Number`、`String`、`Boolean`、`undefined`和`null`这五种基本数据类型之外，所有的值都是以引用的形式存储的。

变量的操作

在做好变量的定义之后，我们就可以对其进行相关的操作了。例如在上面的“Hello World”程序中，`console.log("你好！", name)`这条代码的作用就是将`"你好！"`这个字符串和`name`变量中的内容拼接成一个字符串，并输出在命令行终端中。当然，我们在这里调用的是一个函数，该函数由拼接字符串等一系列基本操作组成，所以想要学好编程，我们首先得要学会编写这些基本操作。

和大多数编程语言一样，JavaScript中的大部分基本操作都是通过操作符来完成的。这里所谓的操作符，通常指的是能对一到两个目标执行某种操作，并返回结果的符号。为了让读者更清晰地理解这一概念，我们先来看一个具体的示例，请在node解释器的交互模式下输入`7 + 8`这条代码，读者应该会看到如下结果。

```
> 7 + 8
15
```

从这一结果，我们大致上可以得到下面这几点信息。

- `+`是一个操作符；
- 该操作是一次加法运算；
- 该操作的目标是`7`和`8` 这两个数字（它们被称为操作数）；
- 该操作的结果为`15`。

当然，现在我们是直接拿了两个数字来当作操作目标，接下来可以改用变量来执行同样的操作，请继续在node解释器的交互模式下输入如下代码：

```
> const a = 7
> const b = 8
> const c = a + b
> c
15
```

在理解了操作符的概念之后，我们紧接着要问的就是JavaScript中到底有多少个操作符？可以执行哪些操作？一般而言，笔者通常会将JavaScript中的操作符分为基本操作符与对象操作符两大类。在这里，我们就先来介绍一下基本操作符。在JavaScript中，我们可以将其支持的基本操作符按其作用分为位运算符、算术运算符、逻辑运算符、关系运算符、字符串操作符与赋值操作符六种，它们主要用于操作一些简单的数据。下面，就让我们看看它们分别是哪一些运算符。

- **位运算符**：众所周知，计算机中的所有数据都是以二进制形式存储的，位运算是一种直接对整数底层存储形式进行操作的运算，它主要作用于32位的正负整数上。下面，我们用表A-3来罗列一下这些运算符及其使用示例：

运算符	中文术语	使用示例	二进制结果	转换成十进制
&	按位与	x = 5 & 1	0001	1
|	按位或	x = 5	1	0101
~	取反	x = ~ 5	1010	-6
^	异或	x = 5 ^ 1	0100	4

运算符	中文术语	使用示例	二进制结果	转换成十进制
<<	左移	x = 5 << 1	1010	10
>>	右移	x = 5 >> 1	0010	2

表A-3 JavaScript中的位运算

- **算术运算符**：算术运算符主要用于基于数字的算术运算。在这里，我们同样用表A-4来列举JavaScript所支持的算术运算符，并提供相应的使用示例（在这里，我们将变量y的值一律赋值为5）。

运算符	中文术语	使用示例	y 的值	x 的值
+	加法	x = y + 2	5	7
-	减法	x = y - 2	5	3
*	乘法	x = y * 2	5	10
/	除法	x = y / 2	5	2.5
%	余数（取模）	x = y % 2	5	1
++（前置）	自增	x = ++y	6	6
++（后置）	自增	x = y++	6	5
--（前置）	自减	x = --y	4	4
--（后置）	自减	x = y--	4	5

表A-4 JavaScript中的算术运算

- **逻辑运算符**：逻辑运算符主要用于布尔类型的计算，确定布尔值之间的逻辑关系。下面，我们继续用表A-5来列举JavaScript所支持的逻辑运算符，并提供相应的使用说明。

运算符	中文术语	使用说明
&&	逻辑与	仅当两边的操作数都为 true 时，结果为 true，否则结果为 false。
||	逻辑或	仅当两边的操作数都为 false 时，结果为 false，否则结果为 true。
!	逻辑非	将 false 转换为 true，true 转换为 false。

表A-5 JavaScript中的逻辑运算

- **关系运算符**：关系运算符有时也被称之为比较运算符，它主要用于判断相关值之间的关系，常用于流程控制的条件判断中。下面是JavaScript中提供的关系运算符，以及它们的使用说明（在这里，我们将变量x的值一律赋值为5）。

运算符	中文术语	使用说明
==	等于	符号两边的值相等时为真
===	恒等于	符号两边的类型和值均相同时为真

运算符	中文术语	使用说明
!=	不等于	符号两边的值不相等时为真
!==	不恒等于	符号两边的类型和值均不相同时为真
>	大于	符号左边的值大于右边时为真
<	小于	符号左边的值小于右边时为真
>=	大于或等于	符号左边的值大于或等于右边时为真
<=	小于或等于	符号左边的值小于或等于右边时为真

表A-6 JavaScript中的关系运算

- **字符操作符**：或许是因为先天就是被设计用来处理HTML文档的关系，基于字符串类型的操作在JavaScript中是较为特殊的一种操作。它的特殊之处主要体现在其变量类型上，在大多数编程语言中，字符串类型通常都属于对象数据类型，采用引用方式来存储，但在JavaScript中，字符串属于基本数据类型，它的数据是直接存储在变量中的。其次，字符串操作并没有单独的操作符，它只是改变了一些现有的算术运算符和关系运算符的定义，然后用它们来执行字符串操作。例如，只要有一个操作目标为字符串类型，`+`操作符执行的就是字符串的拼接操作，即使另一个操作目标不是字符串类型，它也被自动转换为字符串类型。再例如，当操作目标为字符串类型时，`>`、`<`、`=`等这些关系运算符比较的就不是数字的大小，而是字符在字母表中的先后顺序了。例如，我们可以通过执行以下测试来查看一下相关操作符在字符串类型下的作用。

```
> '1' + 1
'11'
> 1 + '1'
'11'
> 1 + '' + 1
'11'
> 'a' > 'b'
false
> 'bat' > 'owl'
false
> 'cd' < 'dvd'
true
> '10' < '9'
true
> '123' == 123
true
> '123' === 123
false
> '123' === '123'
true
```

值得一提的是，从上面所进行的最后三个测试中，我们看到了一个有趣现象，即字符串'123'和数字 123进行==比较时，由于只是单纯的值比较，所以数字被转换成字符串之后，该比较操作就返回true了。但是当他们进行===比较时，比较的除了值还有类型，比较操作符就得返回false了。正因为如此，今后在判断两个字符串变量是否相同时，我们更倾向于建议读者尽可能多地使用===运算符。

- **赋值操作符**：赋值操作符主要用于赋予变量一个指定的值，并让其所在的赋值表达式返回该值（我们会在下一节中介绍赋值表达式的概念）。下面是JavaScript中提供的赋值操作符，以及它们的使用说明（在这里，我们预先设定`x = 10`、`y = 5`）：

运算符	使用示例	示例说明
=	x = y	将x的值赋值给y，结果 x = 5。
+=	x += y	将x + y的值赋值给x，结果 x = 15。
-=	x -= y	将x - y的值赋值给x，结果 x = 5。
*=	x *= y	将x * y的值赋值给x，结果 x = 50。
/=	x /= y	将x / y的值赋值给x，结果 x = 2。
%=	x %= y	将x % y的值赋值给x，结果 x = 0。

表A-7 JavaScript中的赋值操作

在习惯上，笔者通常会将基本操作符之外的所有操作符归类为对象操作符。这些操作符往往都对应着一种特定的对象类型，大致上可以分成以下几类：

- **函数操作符**：例如，`function`操作符用于定义函数，`()`操作符用于调用函数。
- **数组操作符**：例如，`[]`操作符用于索引数组元素，`in`操作符用于判断某元素是否存在于数组中。
- **实例操作符**：例如，`.`操作符用于调用对象实例的方法与属性，`new`操作符用于新建对象实例，`delete`用于删除对象实例。
- **类型操作符**：例如，`typeof`操作符用于查看变量值的数据类型，`instanceof`操作符用于判断某个变量值是否是某个类型的实例。

正是由于这些操作符都与某种类型的对象相关，所以我们更倾向于等到具体介绍相关对象时再来说明它们的使用方法，这里就不再做单独介绍了。

当然了，我们在计算机中实际所执行的操作往往都是由几类不同的操作符共同组合而成的。但是，当几种不同的操作符出现在同一行代码中时，程序会如何决定这些操作的执行顺序呢？理论上，这个问题可以交由操作符的优先级规则来解决。但在实际编程中，我们会选择极力避免写出因多种操作符出现在一起，而导致操作顺序含糊不清的代码，这一方面是为了增加代码的可读性，另一方面也是为了减低编写这类复杂指令时的出错率。所以，人们通常会选择将代码中较为复杂的操作指令拆分成几条简单的指令，譬如对`++a * b + c / d`这样的操作这里，我们通常会这样做。

```
++a;
const x = a * b;
const y = c / d;
x + y;
```

即使我们在特定情况下不得不写一些复杂的运算指令，通常也都会使用一对圆括号来辅助表达自己的意图，譬如，如果我们不想额外增加x和y这两个变量，也可以这样做。

```
++a;
(a * b) + (c / d);
```

这样一来，无论操作符优先级是如何规定的，我们都可以确定圆括号中的操作会被优先执行，这就不存在任何表达上或阅读上的歧义了。当然，圆括号也可以用于改变原本的操作顺序，譬如，如果我们想先执行加法再执行乘除法，就可以这样做。

```
a * (b + c) / d;
```

总而言之，我们需要记住，编程语言归根结底是一门语言，如何清晰、正确地表达意图是最重要的，任何有可能让我们的表达含糊不清，让别人产生困惑做法都应该极力避免，切忌华而不实地炫技。

A.2.3 表达式与语句

众所周知，计算机程序本质上就是一组用某一门编程语言编写而成的指令序列，人类用这门语言表达自己的意图，而计算机则利用这门语言的解释器或编译器理解人类的意图，并将该意图转换成机器指令并执行它。所以，我们程序员的任务就是要学会用编程语言来表述自己的意图。在JavaScript中，表述意图的基本指令单元通常被称为“语句”。我们在上一节中学习的变量和操作符就相当于人类语言中的“名词”和“动词”，而现在就是要学习如何按照自己的意图将这些“名词”和“动词”组织成“语句”了。

和英文语句通常是由多个短语组成的一样，在JavaScript的语句中，表达式就扮演了“短语”的角色。一条语句中通常包含着一个或多个表达式。另外，和我们写完一句话时常用句号来结尾一样，JavaScript语句也通常会以分号结束。但JavaScript的特殊之处在于，即使我们忘了在语句末尾加上分号，解释器在大部分情况下也会自动帮我们加上。这就意味着，在JavaScript中，语句末尾的分号是可以省略的。事实上，如今确实也有一些JavaScript的编码规范，譬如JavaScript Standard Style会建议我们不加分号，他们认为这样做有助于形成一些良好的编码习惯。当然，不加分号也会带来一些麻烦（譬如，如果语句以`(`、`[`符号开头就可能会出现一些不可预料的结果）。总而言之，用不用分号结束语句，完全取决于个人或个人所属团队的喜好。这基本上属于编码风格的问题，我们只需要保持前后一致，确保代码的可读性即可。下面，就让我们从最简单的表达式语句着手，开始学习编写JavaScript语句吧！

A.2.3.1 表达式语句

如前所述，使用语言最根本的目标是要准确而清晰地表达意图。所以编写语句的第一步是要明确自己要表达的内容，可以是执行某个动作，也可以是呈现某个状态。当然，在没有习惯用JavaScript语言表达自己意图之前，不妨先用自己熟悉的人类语言将要表达的内容写出来。举个例子，如果想将 100 元人民币的币值换算成美元，通常我们会这样表达：

- 先获取人民币的币值：100。
- 再取得人民币对美元的汇率：0.1404。
- 将人民币的币值乘以汇率，即为美元币值。

下面，我们用JavaScript语言来翻译一下上面四个短句（当然，我们在这里首先得假设`CNY`、`exRate`和`USD`这三个变量已经完成了定义）。

```
CNY = 100;
exRate = 0.1404;
USD = CNY * exRate;
```

这里出现了三条JavaScript语句。由于这些语句都由一个或两个表达式组成，所以它们所表达的意图是由其中的表达式类型来决定的，而表达式类型则取决于表达式中起最终作用的操作符。例如在这个例子中，前两条语句中只有一个赋值操作符，所以无疑都属于赋值表达式；而第三条语句则由一个乘法运算符和一个赋值操作符共同组成，似乎应该是一个由算术表达式和赋值表达式组合而成的复合表达式，但在习惯上我们是用“起最终作用”的操作符来为表达式归类的，所以它依然属于赋值表达式。

以此类推，我们今后还会遇到算术表达式、关系表达式、逻辑表达式、函数调用表达式或对象操作表达式等执行各类不同操作的表达式，它们都可以直接组成语句。对于这种只包含表达式的语句，我们通常称之为表达式语句，这是JavaScript中最简单的一种语句。

A.2.3.2 复合语句

在JavaScript中，更多语句是由表达式和其他语法元素共同组合而成的，这些语句往往被用于表述比表达式更复杂的意图，我们将其统称为复合语句。在作用上，我们可以将复合语句进一步细分为语句块、条件语句和循环语句三种类型。下面，让我们分别来介绍一下它们。

- **语句块**：在某些情况下，我们会发现自己用若干条语句描述了一个独立于程序其它部分的意图，这时候就可以选择用一堆大括号将这些语句包括在一起构成一个语句块。例如，对于上面这个币值换算操作，我们可以这样做：

  ```
  {
      let CNY, exRate, USD;
      CNY = 100;
      exRate = 0.1404;
      USD = CNY * exRate;
  }
  ```

  这样一来，上面的一条定义语句和三条赋值语句就组成了一个语句块，它在代码解释器眼里是一条语句。这样做的好处是，我们可以在语句块内部创建一个独立的变量命名空间，在该空间中命名的任何变量名都不会影响到空间外的任何地方，我们通常将该空间称作变量的块级作用域。

- **条件语句**：到目前为止，我们所看到的所有程序都是按照语句出现的顺序一路执行到底的，基本没有任何应变能力。如果我们希望自己编写的程序能具备一定的“随机应变”能力，就得让它执行条件判断。在编程语言中，我们用来表述条件判断的语句叫做条件语句。条件语句在编程设计概念中属于流程控制语句中的一种，它的主要作用是根据某一由程序员预先指定的条件来决定是执行还是跳过某部分的语句（这些语句通常被称之为条件分支）。在JavaScript中，条件语句主要有`if`语句和`switch`语句两种。下面，就让我们分别来介绍一下它们。

  - *`if`语句*：`if`语句是编程设计中最常见、最基本的一种流程控制语句，它根据条件分支的多少，可以分为以下三种形式。

    - *单分支形式*：这种形式的if语句只用于指定在某条件成立时需要执行的条件分支，具体语法如下：

      ```
      if ([条件表达式])
          [分支语句]
      ```

- *双分支形式*：这种形式的if语句会同时指定在某条件成立或不成立时需要执行的条件分支，具体语法如下：

```
if ([条件表达式])
    [分支语句]
else
    [分支语句]
```

- *多分支形式*：这种形式的if语句会根据多个条件来决定程序需要执行的条件分支，具体语法如下：

```
if ([条件表达式])
    [分支语句]
else if ([条件表达式])
    [分支语句]
else if ([条件表达式])
    [分支语句]
...
else
[分支语句]
```

在这里，`[条件表达式]`主要会是一些返回布尔类型的表达式，譬如关系表达式、逻辑表达式等，个别时候也会使用到一些返回`null`、`undefined`值的表达式，即如果表达式为`null`或`undefined`，则被认为条件成立。而`[分支语句]`的部分则既可以是简单的表达式语句，也可以是一个由多条语句组成的语句块。下面，我们通过一个具体的例子来演示一下`if`语句的用法。大家都知道，货币的币值通常不会为负数，基于这一点，我们可以对之前币值换算代码做出如下修改：

```
let CNY, USD;
const exRate = 0.1404;
CNY = 200;
if (CNY >= 0) {
    USD = CNY * exRate;
    console.log('换算的美元币值为：', USD);
}
```

在这种情况下，我们只会在`CNY`的值大于或大于`0` 时才会看到书出结果。但是这种做法有个问题，那就是一旦`CNY`的值为负数，我们在执行这段代码时将看不到任何反馈信息，甚至不确定程序是否运行过。为了解决问题，我们要让代码在条件不成立时也输出一条提示信息。

```
let CNY, USD;
const exRate = 0.1404;
CNY = 200;
if (CNY >= 0) {
    USD = CNY * exRate;
```

```
    console.log('换算的美元币值为: ', USD);
} else {
    console.log('人民币的币值不能为负数! ');
}
```

当然，如果我们还想确保exRate也不是负数，也可以继续将代码修改成一个多分支判断。

```
let CNY, USD;
const exRate = -0.1404; // 现在汇率为负值。
CNY = -100;
if (CNY < 0) {
    console.log('人民币的币值不能为负数! ');
} else if (exRate < 0) {
    console.log('人民币对美元的汇率不能为负数! ');
} else {
    USD = CNY * exRate;
    console.log('换算的美元币值为: ', USD);
}
```

- *switch语句*：我们之前用if语句处理多个条件分支时，面对的是不同的条件表达式。但是如果多个条件分支取决于相同的表达式，并且条件分支的数量在三个以上时，就应该要考虑用switch语句来替代if语句了。下面，我们先来看一下switch语句的基本语法：

```
switch ([表达式]) {
case [预估值 1]:
    [分支语句]
    break;
case [预估值 2]:
    [分支语句]
    break;
case [预估值 3]:
    [分支语句]
    break;
...
case [预估值 n]:
    [分支语句]
    break;
default:
    [分支语句];
    break;
}
```

在这里，[表达式]通常会是一个变量，但也可以是其他任何能提供返回值的东西。然后，从[预估值 1]到[预估值 n]则都是程序员预测[表达式]可能会返回的值。在该语句被执行时，程序会拿这些预估值去一一比对[表达式]返回的实际值，如果存在匹配的预估值，就执行该值之后的所有语句。请注意，这里说的是“所有语句”，即匹配预估值后面所有分支中的语句，如果我们只想执行

它当前所在分支的[分支语句]，就必须要在该[分支语句]后面加上一个break跳转语句，跳出switch语句的执行。关于跳转语句，我们后面会详细介绍。

如果程序没有找到任何匹配的预估值，就会执行由关键字default所标记的的分支。当然，和if语句中的else分支一样，这里的default分支也是可选的，如果程序员觉得没有必要，也可以不设置这一分支，在这种情况下，如果程序员没有找到匹配的预估值，switch语句就会什么也不做。说得更具体一点，整个switch语句的执行可细分为以下步骤：

1. 对switch语句[表达式]部分进行求值，并记录结果。
2. 移动到第一个case分支中，将其预估值与步骤1的结果进行比对。
3. 如果步骤2中的比对结果为true，则执行该case分支后面的“所有代码”。
4. 在相关case分支中的语句执行完成之后，如果遇到break语句就直接退出switch语句。
5. 如果步骤2中的比对结果为false，就继续进到下一个case分支中，重复步骤2到5中的操作。
6. 如果直到最后一个case分支，比对结果依然为false，就执行default分支后面的代码。
7. 如果default分支不存在，就直接退出switch语句。

下面，我们同样通过一个具体的例子来演示一下switch语句的用法。大家都知道，switch这个单词的其中一个含义是交换机，这就让人联想到了拨打电话的原理，所以接下来，我们就用switch语句来模拟一个简单的只能打电话给四个人的电话交换机。

```
let number = 1002;

switch (number) {
case 1001:
    console.log('张三');
    break;
case 1002:
    console.log('李四');
    break;
case 1003:
    console.log('王五');
    break;
case 1004:
    console.log('赵六');
    break;
default:
    console.log('你拨打的是空号！');
    break;
}
```

现在，读者可以自行将上述代码更新到一个名为test.js的脚本文件中，然后查看number的值分别为1001、1002、1003、1004和其他任意数字时执行结果，譬如下面是number值为1002和1005时的结果：

```
书稿\Examples\02_studynodejs on master [!?] via v20.5.0
❯ node .\test.js
李四

书稿\Examples\02_studynodejs on master [!?] via v20.5.0
❯ node .\test.js
你拨打的是空号！
```

除此之外，读者也可以将其中任意一个或几个`break`语句注释掉，看看程序的执行会与之前有何不同。

- **循环语句**：在测试上面这些条件语句时，细心的读者可能已经发现了一个问题，那就是这些条件语句都只能执行一次。而我们如果想测试不同的数据，就得去修改代码本身，这种测试方法不仅操作不方便，而且根本没有办法面对海量的测试数据。如果想要解决这个问题，我们就得想办法让程序能根据我们所指定的条件来重复执行某部分的语句，而这就涉及到了编程语言中另一种流程控制语句：循环语句。在JavaScript中，循环语句也主要有两种：`for`语句和`while`语句。下面，就让我们分别来介绍一下它们。

  - *`for`语句*：从使用习惯上来说，`for`语句的整个循环过程通常会由某个被指定的变量来驱动，因此通常更适合被用来描述一些执行次数确定的循环。其具体语法如下：

```
for ([定义循环变量]; [循环条件测试]; [更新循环变量])
    [被循环语句]
```

下面，我们来具体解释一下for语句中的语法单元，在这里：

  - 首先，`[定义循环变量]`所在位置应该是一个带初始化动作的变量定义语句（譬如let i = 0），由于这个变量将被用于驱动整个循环的执行，故而被称之为循环变量。
  - 然后，在`[循环条件测试]`处，我们会设置一个用于测试循环变量的、能返回布尔类型的表达式（譬如`i < 10`），只要该表达式测试的结果`true`，循环就会一直执行下去，该表达式是循环是否继续的判断依据。
  - 接下来，在`[更新循环变量]`处，我们要设置好该循环每重复执行一次之后，循环变量所要做的更新操作（譬如`++i`），通常情况下，循环变量每更新一次，`[循环条件测试]`就应该要更趋向于返回`false`，否则就有可能会导致无限循环。
  - 最后，`[被循环语句]`就是该循环要重复执行的语句，它既可以是简单的表达式语句，也可以是一个由多条语句组成的语句块。例如，如果你想在命令行终端中输出从 0 到 9 这十个数字，就可以像下面这样做。

```
for (let i = 0; i < 10; ++i) {
    console.log('数字: ', i);
}
```

除此之外，`for`语句还有`for-of`和`for-in`两种特殊形式。其中，`for-of`形式是 ES6 新增的一种循环，主要用于遍历数组、集合等数据结构，而`for-in`则主要用于枚举对象的属性。这两种形式的具体语法，我们将会等到具体介绍数组和对象时再来具体说明。

- *while语句*：while语句与for语句相比，最大的区别在于它并没有为循环变量的初始化和更新预留语法单元，其具体语法如下。

```
while ([循环条件测试])
    [被循环语句]
```

如你所见，while语句中的语法单元只有两个：首先，在[循环条件测试]处，我们会设置一个能返回布尔类型的表达式，只要该表达式测试的结果true，循环就会一直执行下去，该表达式是循环是否继续的判断依据；然后，[被循环语句]就是该循环要重复执行的语句，它既可以是简单的表达式语句，也可以是一个由多条语句组成的语句块。这就意味着，while语句具有更高的自由度，它允许程序员更灵活地安排循环的执行方式（当然了，也相对更容易出错），因此在习惯上更适合用来描述一些执行次数不确定的循环操作。例如，如果我们有一个用于逐行读取文本的函数readLine(text)，它要读取的文本是不确定的，我们无法事先知道文本中有多少行内容，但可以确定该函数会在读取完文本之后会返回false，那么，我们就可以像下面这样做。

```
let line = null;
let i = 1;
while (line = readLine(text)) {
    console.log(i + '. ', line);
    i++;
}
```

当然了，如果我们硬要用for循环实现这样的循环，也是可以做到的，而且while循环也可以用来执行循环次数确定的操作。这里只是笔者在使用习惯上所做的建议，仅供读者参考。除此之外，while语句还有一种特殊的形式，我们称之为do-while语句，其具体语法如下。

```
do {
    [被循环语句]
} while ([循环条件测试]);
```

该循环与一般while循环之间最大的不同之处在于，它即使在[循环条件测试]一开始就返回false的情况下，[被循环语句]也至少会被执行一次。例如，对于上面输出文本的循环，如果我们希望在正式内容输出之前无论如何都要先输出一条提示信息，就可以像下面这样做。

```
let line = '下面是目标文件的内容：';
let i = 1;
do {
    console.log(i + '. ', line);
    i++;
} while (line = readLine(text));
```

- **跳转语句**：在程序执行过程中，我们常常会遇到一些需要提前结束当前执行单元（譬如条件语句、循环语句）的特殊情况，这时候就会需要用到一种能让程序直接改变执行位置的语句，我们称之为跳转语

句。下面，就让我们来介绍一下JavaScript 提供的几种跳转语句。

- *break语句*：对于这种跳转语句，我们之前在介绍`switch`语句时就已经遇到过了，它的主要作用就是让程序直接跳出当前正在执行的条件语句与循环语句。例如，如果我们希望之前那个读取文本的循环在遇到空行时就停止读取，就可以像下面这样做。

```
let line = '下面是目标文件的内容：';
let i = 1;
do {
    if (line === '')
        break;
    console.log(i + '. ', line);
    i++;
} while (line = readLine(text));
```

- *continue语句*：这种跳转语句只能被运用在循环语句中，它的作用是停止执行当前这一轮的循环操作，直接进入下一轮循环。例如，如今有很多文本格式是用空行来分割段落的（例如Markdown）。这时候，如果我们觉得遇到空行就直接停止读取的方式不妥当，程序只需不输出空行就可以了，也可以这样做：

```
let line = '下面是目标文件的内容：';
let i = 1;
do {
if (line === '')
    continue;
console.log(i + '. ', line);
i++;
} while (line = readLine(text));
```

- *return语句*：这种跳转语句主要用于终止函数或整个程序的执行，我们将会在稍后介绍函数时再来说明它的具体使用方法。

- *throw语句*：这种跳转语句是异常处理机制的一部分，也主要用于终止函数或整个程序的执行，我们将会在介绍JavaScript的异常处理机制时再来说明它的具体使用方法。

在学习了上述知识之后，读者应该已经可以开始编写一些具有简单功能的程序了。接下来，我们将致力于将它们封装成细粒度更大的语法元素，并重复调用它们，以便实现一些功能更为复杂一点的程序。

A.3 封装的运用

在具体介绍“封装”这个概念，以及它在编程方法论中的意义之前，先让我们来看一段初步实现了电话交换机及其测试功能的示例代码（该示例位于`Examples/02_studynodejs/TelephoneExchange`目录下的`main.js`脚本文件中），具体如下。

```
// 电话交换机及其测试
// 作者：owlman
```

```
// 初步实现
for (let number = 1001; number < 1006; ++number) {
    switch (number) {
        case 1001:
            console.log('张三');
            break;
        case 1002:
            console.log('李四');
            break;
        case 1003:
            console.log('王五');
            break;
        case 1004:
            console.log('赵六');
            break;
        default:
            console.log('你拨打的是空号！ ');
            break;
    }
}
```

之所以要让大家看这段代码，是因为它除了能证明`switch`语句的所有分支可以用循环来进行全覆盖的测试之外，还有另一个作用，就是充当编程方法论中的反面教材。是的，这段代码在编程方法论上几乎可以说是一无是处。除了在功能的实现和测试方法上都做得非常笨拙之外，这段代码更大的问题是完全模糊了不同任务之间的界线。换句话说，整个程序原本应该很清楚地被分为电话交换机的实现代码和测试代码两部分。它们在理论上应该是两个彼此独立的任务，结果前一个任务的代码出现在另一个任务的代码内部。这意味着，无论我们是想更改电话交换机的功能，还是其测试方法，都得修改同一份代码，这种牵一发而动全身的局面对于任何程序的维护都是非常不利的。在编程方法论上，这种任务之间的依赖关系被称之为耦合度，任务之间的耦合度越高，它们所组成的程序就越难以维护。

为了降低程序中各种任务之间的耦合度，程序员们在漫长实践中总结出了一个被称之为过程式编程的编程范式。在这种编程范式中，我们需要先将不同的任务“打包”成各自形成独立的执行单元，然后让这些执行单元以某种特定方式来进行通信。在编程术语上，我们通常会将这个任务“打包”的过程称之为“封装”，它在编程方法论中最大的作用之一就是降低程序代码中不同任务之间的耦合度。

与此同时，一旦各种特定的任务都被“打包”成了独立的执行单元，这些单元被重复使用的难度也会随之下降。譬如，在将电话交换机及其测试程序“打包”两个独立的执行单元之后，我们既可以用测试交换机的执行单元去测试其他电话交换机，也可以用其他测试代码来测试实现当前电话交换机的执行单元，这大大降低了我们重复编写相同代码的可能性，提高了编程效率。

除了将任务操作“打包”成独立的执行单元之外，我们还会根据任务中数据存取的需要，将相关的数据“打包"成不同的数据结构，譬如我们在之前提到过，收集一般性测试数据应该使用便于枚举的数组，存储电话簿则应该使用便于查询关联键/值对的映射集，这些都是基于数据的具体存取需求来设计的数据结构，它们将有助于改善程序的执行效率。除此之外，在被称作面向对象编程、泛型编程等编程范式中，我们还会将数据和操作进一步“打包”在一起，以此来创建由用户定义的新类型，这将极大地扩展编程语言本身的表述能力，这种能力也是以封装为基础的。总而言之，我们可以将封装在编程方法论中的意义总结如下：

- 封装有利于降低不同任务代码之间的耦合度，提高代码的可维护性。
- 封装有利于提高代码被重复使用的机率，增加编程的效率。
- 封装有利于程序对数据的组织和管理，提升程序的执行效率。

- 封装有助于创建用户自定义的新类型，扩展编程语言自身的表述能力。

现在，相信有一些读者已经开始对笔者喋喋不休的长篇大论感到不耐烦了。毕竟，这段论述显然无法让初学者对“封装”这个概念，或者以此为基础展开的编程方法论有多么深入的理解，我们势必还是需要用具体的代码来帮助读者理解它们。但笔者还是认为先在理论上和思维方式上给出一个大致的方向，这对我们接下来要对电话交换机及其测试程序发动的多层次的封装能有一个提纲举领的作用。譬如，现在读者至少应该已经明白了一件事：如果我们想要把上面那个电话交换机测试的程序修改成一个更复杂、更实用的程序，就得从任务的封装开始着手。

A.3.1 封装为函数

在编程术语中，由任务操作封装而成的执行单位通常被称之为“函数（function）”。在大部分编程语言中，函数都是最低层次的封装，它在编程行为中有着非常重要的地位。而具体到了JavaScript中，函数的地位则更为重要，因为在ES6标准发布之前，JavaScript中并没有用于创建用户自定义类型的语法元素，所以无论是过程式编程还是面向对象编程，函数都是绝对的主角。即使在ES6标准新增了`class`等用于创建用户自定义类型的语法支持之后，由于业界存在着大量ES6发布之前的JavaScript项目，无论我们是想继续维护它们，还是将它们升级到ES6，都需要了解更早之前的做法。所以，在这一节中，我们将会对JavaScript中的函数进行详细介绍。

A.3.1.1 函数的定义

在JavaScript中，定义函数的语法主要有以下三种形式。

- **`function`语句形式**： 这种形式的函数定义通常会以`function`关键字开头，然后接下来的语法与Java、C/C++中的函数定义类似，具体语法如下。

  ```
  function [函数名称]([形参列表]) {
      [函数要执行的语句]
  }
  ```

  在这里，`[函数名称]`的命名规范与变量完全相同，我们可以按照之前所学的变量命名规范给函数起任何自己喜欢的名字。通常情况下，函数的名称应该要能反映该函数所要做的事。然后，`[形参列表]`这部分所设定的是该函数希望从外部获取的数据 。最后，`[函数要执行的语句]`部分设置的就是我们要封装在该函数中的任务操作。

- **直接量形式**： 这种形式的函数定义通常会将一个被称作“函数直接量”的值赋值给一个变量，具体语法如下：

  ```
  [定义变量关键字] [变量名] = [函数直接量];
  ```

  首先，`[定义变量关键字]`可以是`let`或`const`，具体使用哪一个将取决于其定义的函数是否可被修改（稍后介绍如何修改函数本身）。接下来，`[变量名]`在此处就等同于`function`语句形式中的`[函数名称]`，它们遵循相同的命名逻辑。最后，`[函数直接量]`的部分在ES6标准发布之前通常会是一个由`function`语句定义的匿名函数，例如像下面这样：

```
[定义变量关键字] [变量名] = function ([形参列表]) {
    [函数要执行的语句]
};
```

如你所见，匿名函数的定义语法除了无需指定[函数名称]之外，其余都与采用function形式定义的具名函数完全相同。也正因为匿名函数是一个没有名称的、具有函数类型的数据值，故而被称作“函数直接量”。在ES6标准发布之后，函数直接量还多了一种被称作“箭头函数”的新形式，所以对于上面的[函数直接量]部分，我们也可以像下面这样写。

```
[定义变量关键字] [变量名] = ([形参列表]) => [函数要执行的语句];
```

- **构造函数形式**： 这种形式的函数定义通常会以调用构造函数，创建对象的方式来呈现，具体语法如下：

```
[定义变量关键字] [变量名] = new Function()('[形参列表]', '[函数要执行的语句]');
```

关于构造函数的使用，我们将会在下一节中介绍“对象”这个概念时再做说明。在这里，读者只需要知道这种形式的存在，并且函数在JavaScript中也属于对象即可。而且，出于代码的可读性和可维护性方面的考虑，我们通常是不建议使用这种形式来定义函数的。

值得一提的是，JavaScript和Java、C/C++这些语言不同，它的函数是不必特别指定返回值类型和形参类型的，其返回值类型将由函数体内的return语句来决定，而形参类型则由传递给该函数的实参来决定。

A.3.1.2 函数的调用

在编程术语中，将封装在函数中的代码运行起来的操作通常被称作“调用（call）”。和大多数编程语言一样，在JavaScript中，这一动作也是通过函数调用表达式来完成的，其语法具体如下。

```
[函数名称]([实参列表]);
```

在这里，[函数名称]必须是已经被定义的函数或引用了函数直接量的变量。而[实参列表]则是调用方传递给被调用函数的数据。在通常情况下，函数调用方提供的[实参列表]应该与被调用函数所声明的[形参列表]应该是一一对应的。下面，我们可以通过编写一段示例代码来演示一下函数的定义与调用语法了，具体如下。

```
function add_1 (x,y) {                          // function语句形式
    return x + y;
}

const add_2 = function(x,y) {                   // 直接量形式1
    return x + y;
};

const add_3 = (x,y) => {                        // 直接量形式2
    return x + y;
```

```
};

const add_4 = new Function('x,y', 'return x + y;'); // 构造函数形式

console.log(add_1(4,7));
console.log(add_2(4,7));
console.log(add_3(4,7));
console.log(add_4(4,7));
```

读者如果将上述代码更新到`test.js`脚本文件中，并执行一下看看结果，会看到上面四个函数虽然使用了不同的定义语法，但执行之后的输出是完全相同的。

A.3.1.3 函数就是值

到目前为止，我们介绍的还都是函数的一般用法，它们看起来与Java、 C/C++等语言中的函数并没有太大的区别，但在JavaScript中，函数事实上是一种非常特殊的存在。从某种程度上来说，我们能否很好地驾驭这门语言就取决于能否随心所欲地使用它的函数。函数之所以在JavaScript中享有如此特殊的地位，主要是因为它在JavaScript中不仅仅是一种封装了代码的执行单元，同时也是一种数据类型的值。也就是说，在JavaScript中，函数可以和数字、字符串一样被当作数据赋值给变量、被当作实参传递给另一个函数、以及被别的函数当作返回值传回来，这赋予了这门语言极大的灵活性，譬如对于下面示例中的`printArguments`函数，我们也可以用一个变量来调用它。

```
function printArguments() {
    for (let i = 0; i < arguments.length; ++i) {
        console.log(arguments[i]);
    }
}

const caller = printArguments;

printArguments('one','two','three');
caller('batman','owlman');
```

从上述代码中，我们很容易看出来，函数在JavaScript中是作为引用值存储在变量中的，`caller`相当于是指向`printArguments`函数的指针，它们执行的是同一份代码。接下来，为了让读者更深入地理解这种灵活性，我们要回到之前那个实现了电话交换机及其测试的程序中，研究一下如何将电话交换机的实现代码和测试代码分离出来，各自封装成一个函数。这其中，对电话交换机实现部分的封装是相对简单的，它只是一条`switch`语句，并且需要一个数字类型的数据作为参数，具体代码如下。

```
function telephoneExchange(number) {
    switch (number) {
    case 1001:
        console.log('张三');
        break;
    case 1002:
        console.log('李四');
        break;
```

```
    case 1003:
        console.log('王五');
        break;
    case 1004:
        console.log('赵六');
        break;
    default:
        console.log('你拨打的是空号！');
        break;
    }
}
```

然后测试代码的函数应该怎么写呢？有些人可能会这样做：

```
function testTelephoneExchange () {
    for (let number = 1001; number < 1006; ++number) {
        telephoneExchange(number);
    }
}
```

这样做的确也可以完成任务，代码执行的结果也是正确的，但它并没有解决我们之前提出的耦合度问题，一旦有人觉得交换机的实现函数`telephoneExchange`名字过长，想修改函数名，他就必须同步修改其测试函数`testTelephoneExchange`中的调用操作。而且，如果测试函数想测试一下别的交换机实现，它也得修改自身的代码，这大大影响了该函数被重复使用的可能性。那么，应该怎么做呢？我们可以先想一个问题：测试函数中有什么数据是应该由调用方来提供的呢？难道不正是它要测试的函数吗？毕竟从逻辑上来说，只有测试函数的调用者才会知道它要测试的是哪个函数。所以，被测试的函数应该是个参数，由调用方来提供。这样一来，对于这个程序的解耦合工作，我们最终处理的结果应该如下所示。

```
function telephoneExchange(number) {
  switch (number) {
    case 1001:
      console.log('张三');
      break;
    case 1002:
      console.log('李四');
      break;
    case 1003:
      console.log('王五');
      break;
    case 1004:
      console.log('赵六');
      break;
    default:
      console.log('你拨打的是空号！');
      break;
  }
}
```

```
function testTelephoneExchange (callback) {
  for (let number = 1001; number < 1006; ++number) {
    callback(number);
  }
}

testTelephoneExchange(telephoneExchange);
```

这样一来，如果再有人想修改`telephoneExchange`函数的名字，就只需要告诉测试函数的调用方即可，无需再修改`testTelephoneExchange`函数了。另外，如果测试函数的调用方想测试一个临时的交换机实现方案，我们也可以写一个函数直接量，并将其作为实参传递给`testTelephoneExchange`函数，像这样：

```
testTelephoneExchange( number => {
    if (number == 1001) {
        console.log('batman');
    } else if (number == 1002) {
        console.log('owlman');
    } else {
        console.log('你拨打的是空号！ ')
    }
    });
```

在编程术语上，对于这种让函数执行其调用方提供的函数的操作，我们通常称之为回调（callback），今后读者会看到，这是JavaScript中非常常见的函数使用方法，它是这门语言之所以强大的重要原因之一。

A.3.1.4 使用内置函数

在程序员群体中，时常流行着一些指导原则，这些原则能让我们的编程行为更有的放矢，事半功倍。在这里，我们先来讨论一下“不要重复发明轮子”这条原则。该原则的意思是，在开始编程之前，程序员应该先搞清楚自己所在的编程环境和手里拥有的工具，只要有现成的东西可用，就应该尽量避免自己去创建一个一模一样的东西。譬如ECMAScript标准为我们定义了一些内置函数，这些函数是各个版本的JavaScript实现都应该要提供的，所以我们在编写JavaScript代码之前，应该要了解一下这些函数，以避免“重复发明”它们。这些内置函数包括。

- **parseInt()函数**：该函数主要用于将其收到的实参值（通常是字符串）转换成整数类型输出。如果转换失败就返回`NaN`。
- **parseFloat()函数**：该函数的功能与`parseInt()`类似，只不过它转换的是浮点数。
- **`isNaN()`函数**：该函数主要用于确定某个输入值是否是一个可以参与算术运算的数字。
- **`isFinite()`函数**：`isFinite()`函数与`isNaN()`函数相对，主要用于判断某个数字类型的值是否是一个既非`Infinity`也非`NaN`的具体数字。
- **`eval()函数**：该函数是一个相当奇特的存在，它的主要作用是将其接收到的字符串实参当做JavaScript代码来执行。当然，如果有选择的话，我们在大多数情况下都应该尽量避免使用这个函数，对于许多经验丰富的JavaScript程序员来说，“Eval is evil”（Eval是魔鬼）是一句至理名言，其主要原因如下。
    - 执行安全方面：由于JavaScript的使用非常自由，且功能很强大，这也同时带来了很大的不确定性，非常考验程序员自身的能力，所以，如果我们对放在`eval()`函数中的代码没有太多把握，最好不要冒无谓的风险。
    - 执行性能方面：由于这是一种由函数“动态”执行的代码，所以要比直接执行脚本要慢。

- **编码与反编码函数**：在URL（即Uniform Resource Locator，统一资源定位符）或URI（即Uniform Resource Identifier，统一资源标识符）中，有一些字符是具有特殊含义的。如果我们想“转义”这些字符，就可以去调用`encodeURI()`或`encodeURIComponent()`这两个函数。当然，`encodeURI()`和`encodeURIComponent()`也都有各自对应的反编码函数：`decodeURI()`和`decodeURIComponent()`。

A.3.2 封装为对象

在上一节中，我们通过将不同的任务操作封装成独立的函数，解决了代码中的高耦合度问题。接下来，我们将注意力转向任务本身的实现，下面仍以之前的电话交换机程序为例，一直到目前为止，我们是这样实现它的。

```
function telephoneExchange(number) {
    switch (number) {
        case 1001:
            console.log('张三');
            break;
        case 1002:
            console.log('李四');
            break;
        case 1003:
            console.log('王五');
            break;
        case 1004:
            console.log('赵六');
            break;
        default:
            console.log('你拨打的是空号！ ');
            break;
    }
}
```

上述实现的另一个问题是：与实际需求脱节。毕竟在现实环境中，只有四条线路的电话交换机是没有任何实际意义的，哪怕就是一家公司的内部电话网，通常也有几十条甚至上百条电话线路呢，而且任何电话网络都要有随时增加或移除线路的需求。所以，无论从哪个角度看，用一条`switch`语句来实现电话交换机都不是一个理想的做法。我们应该将电话线路存储到一个可随时增加、修改、查询和删除的“电话簿”中，然后在让程序自动化管理它。从编程方法上来说，如果我们想根据特定的任务需要来安排数据，实现某种程度的自动化管理，最好的方式就是将执行这些任务的函数与其相关的数据进一步封装成一个整体。

A.3.2.1 对象的定义

在编程术语上，我们通常将由函数与其相关数据封装而成的整体称之为“对象（object）”，被封装在对象中的函数被称作“对象的方法”，数据则被称作“对象的属性”。譬如，在之前介绍函数的时候，我们曾经用过`arguments`这个对象来获取当前函数收到的实参数据，其`length`属性中存储的就是函数收到的实参个数。

在JavaScript中，对象的定义是动态的，其对象的属性和方法是可以随时根据编程需要增加、修改和删除的，这一点和Java这类传统面向对象编程语言先定义类再定义对象的做法有很大的不同。甚至在ES6标准发布之前，JavaScript中压根就不存在类定义的语法形式。下面，我们就来具体介绍一下在JavaScript中定义对象的几种不同形式：

- **直接量形式**： 这种形式会以字面直接量的形式列出对象的属性和方法，然后将它赋值给一个变量，具体语法如下。

```
[定义变量关键字] [对象名] = [对象直接量];
```

首先，`[定义变量关键字]`可以是`let`和`const`中的任何一个，至于具体选择哪一个，就得取决于这个对象本身是否可被修改。然后，`[对象名]`是我们给该对象起的名称，它的命名逻辑与一般变量。最后，`[对象直接量]`的部分应该是一个由大括号括起来的属性及方法列表，彼此之间用逗号分隔。例如，如果我们想创建一个拥有二维坐标的“点”对象，就可以像下面这样做。

```
let pointA = {
    x : 10,
    y : 10,
    printCoords : function () {
        console.log('坐标:  ('+ this.x + ', ' + this.y + ') ');
    }
};
```

如你所见，我们在创建`pointA`这个对象的同时，为它设置了`x`和`y`两个属性，以及一个名为`printCoords`方法。由于在JavaScript中，函数本身也是一种值，所以我们可以将对象中的每个属性或方法都看作一对名/值组合，名与值之间用冒号分割，而整个对象就是一系列名/值对的映射集。

- **构造函数形式**： 这种对象创建形式大部分语法元素与直接量形式相同，只不过这回对象的直接量变成了一个用`new`操作符调用的构造函数。在JavaScript中，所有的对象类型都会有一个构造函数，其主要作用是为新建的对象分配内存，并将其初始化。它们必须经由`new`操作符来调用，具体语法如下。

```
[定义变量关键字] [对象名] = new [构造函数]([构造实参]);
```

当然，在这么做之前，我们得先定义好对象的构造函数。在JavaScript中，构造函数的定义与一般函数并没有太大的不同，只不过在约定俗成上，我们通常会让构造函数的首字母大写。譬如，对于之前的`pointA`对象，我们也可以这样定义。

```
function Point(x,y) {
    this.x = x;
    this.y = y;
    this.printCoords = function () {
        console.log('坐标:  ('+ this.x + ', ' + this.y + ') ');
    };
}

let pointA = new Point(10,10);
let pointB = new Point(20,20); // 构造函数可以用来定义多个对象。
```

如你所见，构造函数似乎还可以充当传统编程语言中类的角色，即我们可以用它来定义多个同类对象。但请千万不要误会，尽管在ES6标准之前的JavaScript代码中，我们的确会在实际使用中让构造函数来充当类的角色，但该函数本身自始至终都只是一个普通函数，使其发挥对象构造作用的是`new`操作符。而且，当构造函数被当作普通函数调用时，其中的`this`关键字往往会带来一些意想不到的效果，稍后我们会专门讨论一下这个问题。

- **类定义形式**： 这种对象创建形式是ES6标准新增的，它采用的是我们在Java这些传统面向对象语言中所熟悉的做法，先定义类，然后再用类去创建对象，具体语法如下。

```
class [类名] {
    constructor([构造形参]) {
        [创建属性并将其初始化]
    }

    [方法定义]
};

[定义变量关键字] [对象名] = new [类名]([构造实参]);
```

对于学习过Java这些语言的读者来说，这种语法是不是让人很有亲切感？甚至凭直觉就能使用它了。譬如，如果我们想把之前的`Point`构造函数改写成类的形式，就可以这样做：

```
class Point {
    constructor(x,y) {
        this.x = x;
        this.y = y;
    }

    printCoords() {
        console.log('坐标：  ('+ this.x + ', ' + this.y + ') ');
    }
};

let pointA = new Point(30,30);
```

如果仔细对比一下类定义形式与之前的构造函数形式，就会发现两者在语法元素上相差无几。但ES6标准新增的这个类定义形式，解决了构造函数可能给我们带来的两个隐患：第一，它将构造函数封装在类定义中，这样就从根本上杜绝了它被当作普通函数调用的可能性。第二，之前我们在构造函数中定义方法时，实际上采用的是与属性相同的方式，这意味着用构造函数创建的每个对象都存有一份独立的方法定义。但属性与方法在语义上是非常不一样的，属性定义的是每个对象的个体状态，它们会随着对象的不同而不同，但方法定义的是同一类对象都会执行的操作。如果创建一百个对象，相同的方法就要被重复定义一百次，这显然会造成毫无必要的代码冗余。

当然了，我们在这里还必须要解释一个问题，即当构造函数被不小心当作普通函数来调用时到底会发生什么？这个问题主要和JavaScript中的`this`关键字有关。在其它编程语言中，`this`关键字通常只能用在类的定义中，指代的是被定义的属性或方法所在的类。但在JavaScript中，`this`关键字可以出现在任何函数中，它指代的是函

数的调用方。至于究竟谁是函数的调用方，网页浏览器和Node.js等环境又有不同的实现（尤其是对于全局对象的理解），读者可以自行在各种环境中测试一下如下代码：

```
let name = '全局';

function myFunc () {
    console.log(this.name);
}

function test() {
    let name = 'test函数';
    myFunc();
}

myFunc();
test();
```

不管读者在测试中看到了什么执行结果，可以肯定的是，这些结果在大部分情况下不会是我们想要的，甚至很多时候是难以理解的。所以现在知道不小心把构造函数当成普通函数来调用会有什么后果了吗？答案是没有人可以准确预料，而对于程序员来说，没有什么情况比这更糟糕了。要想避免这个问题，我们通常会在构造函数中用`instanceof`操作符判断一下`this`是否为指定类型的对象，例如像下面这样。

```
function myFunc () {
    if (this instanceof myFunc) {
        this.name = 'owlman';       // 已经用 new 调用，正常执行构造函数代码
    } else {
        return new myFunc();        // 没有用 new 调用，就用 new 调用一下自己
  }
}

let obj = myFunc();
console.log(obj.name);          // 输出：owlman
```

当然，除了期望自己小心之外，最好的办法是将代码声明为“严格模式”，让JavaScript解释器帮我们杜绝这种情况，具体做法如下。

```
'use strict';    // 声明为严格模式

let name = '全局';

function MyFunc () {
    console.log(this.name);
}

MyFunc();
```

这样一来，当我们执行上述代码的时候，JavaScript解释器就会报错。

在介绍完对象的定义和使用之后，我们就可以回到电话交换机的实现问题上来了。该如何定义一个“电话交换机”对象呢？我们之前已经分析过了，它首先应该有个可供快速查询的“电话簿”，这应该被设为该对象的属性，然后围绕着该“电话簿”展开的增加、删除、修改和查询等操作则设为该对象的方法。所以，接下来的任务就是要找到实现“电话簿”的有效方案。

A.3.2.2 数据结构对象

在编程术语中，我们通常会将类似“电话簿”这种的，根据任务需求来组织数据并使其结构化的存储形式称作数据结构（data structure）。换句话说，数据结构的组织逻辑取决于其内部数据将要执行的操作，例如，适用于遍历、枚举的数据组织逻辑与适用于快速查找的数据组织逻辑是完全不同的，适用于频繁增加/删除数据的又是另一种组织逻辑，它们通常都对应着不同的数据结构。数据结构的分类有很多种不同的方式，譬如按照实现方式可分为数组、链表、散列表、树结构、图结构等，按照功能分可分为列表、堆栈、队列、集合、映射等，如果要详细介绍的话，我们可能需要另外写一本书。但由于JavaScript是一门高级编程语言，我们并不需要太了解数据结构的具体实现，只需要聚焦于自己的任务需求与数据结构功能之间的关联即可。在日常编程需求中，较为常用的数据结构主要有以下三种。

- **列表（List）**：这种数据结构既可以是一个由整块连续内存空间组成的数组，也可以是由一系列动态分配内存串联而成的链表，并可根据需要进一步封装成堆栈、队列等更为专用的数据结构。它主要适用于遍历、枚举数据。
- **集合（Set）**：这种数据结构主要用于存储一组不重复的数据，由于它常用树结构来组织数据，所以通常适用于排除重复数据，以及按某种优先策略对数据进行排序。
- **映射集（Map）**：这种数据结构又是也被称作字典，主要用于存储具有键/值对特征的数据。在该数据结构中，充当“键”的部分往往会是一些便于查询的数据，目的是让我们通过“键”来查找其对应的“值”，所以这是一种适用于快速查询到数据结构，通常会采用散列表的结构来实现。

在学习C这类接近计算机底层的编程语言时，我们通常需要亲自动手来实现数据结构，这会耗费大量的时间和精力。幸运的是，在JavaScript中，尤其在ES6标准发布之后，这门语言对于上面提到的三种常用的数据结构都提供了对应的内置对象，我们就不必“重复发明轮子”了。下面就先来介绍一下这些数据结构型的内置对象。

列表类对象

在JavaScript中，列表、堆栈、队列这一类线性数据结构的功能通常是由`Array`对象来提供的，创建该对象的形式也有直接量和构造函数两种。

- **直接量形式**：这种形式会以字面直接量的形式列出列表中的元素，然后将它赋值给一个变量，语法如下：

  ```
  [定义变量关键字] [列表名] = [列表直接量];
  ```

  在这里，`[定义变量关键字]`可以是`let`和`const`中的任何一个，至于具体选择哪一个，就得取决于我们是否允许该列表被后面的代码修改。然后，`[列表名]`是我们给该列表起的名称，它遵循与一般变量相同的命名逻辑。最后，`[列表直接量]`的部分应该是一个由中括号括起来的一组值，各值之间用逗号分隔，我们称这些值为列表的元素。例如，如果我们想创建一个包含了从1到5的整数值列表，就可以像下面这样做。

```
const arr = [1, 2, 3, 4, 5];
```

- **构造函数形式**： 这种列表创建形式是用new操作符去调用Array对象的构造函数，具体语法如下。

  ```
  [定义变量关键字] [数组名] = new Array([构造实参]);
  ```

  如你所见，这种形式中的大部分语法元素与直接量形式相同，只不过这回直接量部分变成了一个用new操作符调用的构造函数。另外，我们还可以在调用构造函数的时候指定一些[构造实参]，这些参数既可以是一个代表元素数量的数字，例如，如果我们想定义一个拥有三个元素的列表，就可以像下面这样做。

  ```
  const threeNumbers = new Array(3);
  ```

  也可以是用于初始化列表的一组元素。例如，对于之前用的数组arr，我们也可以这样定义：

  ```
  const arr = new Array(1, 2, 3, 4, 5);
  ```

在定义完列表之后，我们就可以对它执行相关的操作了。首先，我们可以通过Array对象的length属性来了解列表中当前有多少元素，然后就可以用一对中括号加一个非负的整数值来访问列表中的元素了，例如下面这样。

```
let arr = new Array(1, 2, 3, 4, 5);

for(let i = 0; i < arr.length; ++i) {
    console.log(arr[i]);   // 逐行输出： 1, 2, 3, 4, 5
}
```

在编程术语上，像arr[i]这样用一个非负整数来访问列表元素的操作被称作“索引”，这里的i代表的是被索引元素在列表中的位置，被称为“索引值”，它应该是一个从0开始计数的非整数值。而像上面代码一样将列表中的元素从头到尾索引一遍的操作，我们称之为“遍历”操作。对于遍历操作， ES6标准为我们新增了一种新的for循环形式，例如，对于上面的列表遍历，我们也可以换成下面这种写法。

```
const arr = new Array(1, 2, 3, 4, 5);

for(let iter of arr) {
    console.log(iter);   // 逐行输出： 1, 2, 3, 4, 5
}
```

在for-of这种循环形式中，程序会依次将arr列表中的元素读取到循环变量（即这里的iter变量）中。这样做的好处在于，我们不必事先去读取列表的length属性值，并担心索引值在某种意外情况下突然大于该值，出现

“越界访问”的情况。另外，`Array`对象作为一个对象类型，自然还提供了其他一系列操作线性数据结构的内置方法，以下是ES6标准发布之前就很常用的方法。

- `push`方法：该方法主要用于在列表的末端添加新的元素。
- `pop`方法：该方法的功能正好与`push`方法相反，主要用于删除列表末端的元素。
- `unshift`方法：该方法主要用于在列表的前端添加新的元素
- `shift`方法：该方法的功能正好与`unshift`方法相反，主要用于删除列表前段端的元素。
- `join`方法：该方法主要用于按照其实参指定的分隔符将列表中所有元素串联成一个字符串，如果调用方没有提供实参，则使用逗号为默认分隔符。
- `sort`方法：该方法主要用于按照调用方提供的回调函数实参对列表中的元素进行排序，该函数返回负数表示小于，返回 0 表示等于，返回正数表示大于。如果调用方没有提供实参，则使用字母表的先后顺序为默认排序规则。如果列表中存在值为`undefined`的元素，则一律被放在列表中其他元素的后面。
- `reverse`方法：该方法主要用于反转列表中元素的排列顺序。
- `concat`方法：该方法主要用于将当前列表与其实参指定的一个或多个列表连接起来，并作为新的列表返回给调用方。
- `slice`方法：该方法主要用于截取当前列表的某个由实参指定的子列表。通常情况下，调用方会用两个实参指定子列表的开始位置和结束位置。如果只指定了一个位置，这就截取从该位置开始一直到当前列表末端的所有部分。如果调用方提供的实参为负数，则该实参指定的位置索引值为数组的长度减去该实参的绝对值。
- `splice`方法：该方法主要用于从当前列表中删除或替换由实参指定的子列表。在执行删除任务时，，调用方会用两个实参指定子列表开始位置和结束位置的索引值。如果只指定了一个索引值，则就删除从该位置开始一直到当前列表末端的所有部分。如果调用方提供的实参为负数，则该实参指定的位置索引值为数组的长度减去该实参的绝对值。如果要执行替换任务，调用方就会提供三个及三个以上的实参，并从第三个实参起，列出要插入在被删除位置的元素。
- `toString`方法：该方法主要用于将当前列表中的所有元素以逗号为分隔符连接成一个字符串，并返回。它的功能与不带实参调用的`join`方法是相同的。

下面，我们就来演示一下这些方法的使用：

```
// 使用 Array 对象实现堆栈
let stack = [];
for (let i = 0; i < 3; ++i) {
  stack.push(i);
}
console.log(stack.join());    // 输出： 0, 1, 2
while(stack.length) {
    console.log(stack.pop());   // 逐行输出： 2, 1, 0
}                              // 体现堆栈“先进后出”的特性

// 使用 Array 对象实现队列
let queue = [];
for (let i = 3; i < 6; ++i) {
    queue.unshift(i);
}
console.log(queue.join());    // 输出： 5, 4, 3
while(queue.length) {
    console.log(queue.pop()); // 逐行输出： 3, 4, 5
}                              // 体现队列“先进先出”的特性
```

```
// 其他列表操作
let list = [0,1,2].concat([5,4,3]); // 将两个列表合并成一个列表
console.log(list.join());           // 输出: 0, 1, 2, 5, 4, 3
console.log(list.slice(2,5));       // 输出: [2, 5, 4] 子序列
list.splice(0,1,6,7,8);             // 将 0 删除，替换成 6, 7, 8 三个元素
console.log(list.join());           // 输出: 6, 7, 8, 1, 2, 5, 4, 3
list.push(10);
list.sort();                        // 按字典顺序排序
console.log(list.join());           // 所以 10 排在 2 之前
list.sort((x, y) => {
    return x - y;
});                                 // 按数字大小排序
console.log(list.join());           // 所以 10 排在最后
console.log(list.reverse());        // 反转列表中的元素顺序
```

在ES6标准发布之后，`Array`对象得到了一系列扩展。下面，让我们接着来介绍一些其中较为常用的新方法。

- 遍历运算符： 该运算符的用法就是在一个`Array`对象之前加三个点，即...[Array对象]，这样它就会自动遍历`Array`对象，并逐一返回其中的元素。也就是说，我们现在可以像下面这样来输出Array对象中的所有元素。

  ```
  console.log(...[1,2,3]); // 输出 1 2 3
  // 这意味着，我们执行某些数组操作变得更方便了，例如:
  const arr1 = [1,2,3];
  const arr2 = [...arr1];              // 深拷贝 Array 对象
  arr1.push(...[5,6,7]);               // 一次性追加多个元素
  console.log(...arr1, ...arr2);       // 合并列表
  ```

- `Array.from()`方法： 该方法主要用于将其他类数组对象或后面要介绍的集合类对象与字典类对象转换成`Array`对象，例如像下面这样。

```
// 先定义一个类数组对象:
const arrayLike = {
    '0': 'a',
    '1': 'b',
    '2': 'c',
    length: 3
};

// 再将其转换成真正的 Array 对象:
console.log(Array.from(arrayLike)); // 输出 ['a', 'b', 'c']
```

- `Array.of()`方法： 该方法主要用于将一组值转换成数组，我们通常会利用该方法将一组现有变量转化为`Array`对象，例如像下面这样。

```
const a = 4;
const b = a + 5;
```

```
const c = b - 2;
console.log(Array.of(a,b,c)); // 输出 [4, 9, 7]
```

- `find()`方法： 该方法主要用于查找`Array`对象中第一个匹配指定条件的元素，它接收一个函数类型的实参，用于指定匹配条件，例如像下面这样。

```
const arr3 = [1,2,3,4,5,6].find(function(value) {
    return value > 3;
});
console.log(arr3); // 输出 4，这是数组中第一个大于 3 的数字。
```

- `findIndex()`方法： 该方法的作用与使用方式与`find()`方法基本相同，唯一的区别是，它返回的是被查找元素在数组中的索引值。

需要提醒读者的是，这里介绍的只是笔者个人认为较为常用的`Array`方法。出于篇幅方面的考虑，对于一些不那么常用的方法，我们就不针对`Array`对象来做特别介绍了。例如，ES6为了统一容器类型对象的遍历操作而为`Array`对象增加的`Iterator`接口，这些接口在`Array`对象本身的使用中并不常见，所以我们打算在后面讨论集合类对象和字典类对象时介绍这些`Iterator`接口，毕竟到那时候这些接口的作用会更好理解一些。如果读者想更全面地了解`Array`对象提供的接口，可自行查阅ES6的标准文档。

集合类对象

在ES5及其之前的标准中，JavaScript标准库中并不提供可实现集合类数据结构的内置对象，但在ES6标准发布之后，我们可以用`Set`对象和`WeakSet`对象来创建集合类数据结构了，这类数据结构通常会以树结构的形式来存储数据，主要用于存储一组不重复的元素。下面，我们先来介绍`Set`对象。

`Set`对象

在JavaScript中，`Set`对象只能透过其构造函数来创建，其具体语法如下。

```
[定义变量关键字] [集合名] = new Set([构造实参]);
```

如你所见，`Set`对象的构造语法与`Array`对象的构造函数形式基本相同。只不过在这里，`[构造实参]`部分依然会是一个现有的`Array`对象或者其直接量。例如：

```
// 使用现有的 Array 对象来构造 Set 对象
let arr = [1, 2, 3, 4, 3, 4, 5];
let iset = new Set(arr);

// 或者使用 Array 对象的直接量来构造 Set 对象
let mset = new Set([1, 2, 2, 3, 4, 5]);

console.log(iset);   // 输出：Set { 1, 2, 3, 4, 5 }
console.log(mset);   // 输出：Set { 1, 2, 3, 4, 5 }
```

如你所见，即使用来创建集合的列表有所不同，但在消除了重复元素之后，`iset`和`mset`这两个对象中的元素是相同的。在定义完集合之后，我们就可以对它执行相关的操作了。首先，我们可以通过`Set`对象的`size`属性来了解该集合中当前有多少元素，例如，对于上面的集合对象，我们可直接通过`iset.size`这个操作来获取`iset`对象中的元素个数。另外，我们也可以用`for-of`循环来遍历`Set`对象：

```
let iset = new Set([1, 2, 3, 4, 3, 4, 5]);

console.log(iset.size); // 输出： 5
for(let el of iset) {
    console.log(el);      // 逐行输出： 1, 2, 3, 4, 5
}
```

另外，`Set`对象作为一个对象类型，自然还提供了其他一系列操作集合类数据结构的内置方法：

- `add`方法：该方法用于往集合中添加元素，如果被添加的元素已经存在于集合中，则添加失败。
- `delete`方法：该方法用于从集合中删除元素。
- `has`方法：该方法用于判断某个元素是否存在于集合中。
- `clear`方法：该方法用于清空集合中的所有元素。
- `values`方法：该方法会以一个遍历器的形式返回集合中的所有元素。
- `forEach`方法：该方法用于遍历集合，需要调用方指定一个回调函数的实参，用以执行循环操作。

下面，我们就来演示一下这些方法的使用：

```
// Set 对象操作
let mySet = new Set();       // 创建一个空集合
for(let i = 0; i < 5; ++i) {
    mySet.add(i);              // 添加五个元素
}

console.log(mySet.has(4));   // 输出： true
mySet.delete(4);
console.log(mySet.has(4));   // 输出： false
mySet.forEach(value => {
    console.log(value);        // 逐行输出 0, 1, 2, 3
});
console.log(mySet.values()); // 输出： SetIterator { 0, 1, 2, 3 }
```

`WeakSet`对象

顾名思义，这是一个弱化版的`Set`对象，它的大部分使用方式都与`Set`对象相同，这里就不再复述了。下面我们来关心一下它们的不同之处。

- 首先，WeakSet对象中只能存储对象类型的引用，不能存储基本类型的值。这意味着，对于如下代码，解释器是会报错的。

  ```
  let wset = new WeakSet();
  wset.add(10);   // 报错
  ```

- 其次，`WeakSet`对象中存储的元素都是弱引用对象。也就是说，JavaScript的垃圾回收机制不会将任何对象存储在`WeakSet`对象中的引用计算在内。这意味着，如果某个对象已经没有其他地方引用了，垃圾回收机制就会自动回收该对象所占用的内存，无论它是否还有引用存储在`WeakSet`对象中。

基于以上特点，`WeakSet`对象中的元素是不适合被引用的，因为它会随时消失。另外，由于`WeakSet`对象中的有效元素数量某种程度上取决于垃圾回收机制有没有运行，其元素数量也是不可预测的。因此ES6标准建议将`WeakSet`对象视为不可遍历的集合，也就是说，`Set`对象中用于遍历的`values`、`forEach`等方法，`WeakSet`对象均不支持。

字典类对象

从本质上来说，JavaScript中的所有对象都是一组键/值对的集合，只不过它们通常只能使用字符串类型的“键”，这给相关数据结构的使用带来了很大的限制。为了解决这个问题，ES6为我们提供了专用于存储键/值对的`Map`对象和`WeakMap`对象。这样一来，我们今后再存储键/值对类型数据时，“键”的类型就可以不限于字符串类型了，这有利于我们构建更强大的键/值查询。例如，之前提到的“电话簿”，就是一个以数字类型的“键”来查询人名（字符串）的结构。下面，我们照例先来介绍一下`Map`对象。

`Map`对象

在JavaScript中，`Map`对象也只能透过其构造函数来创建，其具体语法如下。

```
[定义变量关键字] [字典名] = new Map([构造实参]);
```

如你所见，`Map`对象的构造语法与`Set`对象基本相同。只不过在这里，`[构造实参]`部分是一个二维的`Array`对象或其直接量。例如：

```
// 使用现有的二维 Array 对象来构造 Map 对象
let arr2d = [[1,'owl'], [2,'bat'],[3,'cat'], [4,'dog']];
let imap = new Map(arr2d);

// 或者使用二维 Array 对象的直接量来构造 Map 对象
let mmap = new Map([[1,'owl'], [2,'bat'],[3,'cat'], [4,'dog']]);

console.log(imap);// 输出：Map { 1 => 'owl', 2 => 'bat', 3 => 'cat', 4 => 'dog' }
console.log(mmap);// 输出：Map { 1 => 'owl', 2 => 'bat', 3 => 'cat', 4 => 'dog' }
```

在定义完字典之后，我们就可以对它执行相关的操作了。首先，我们可以通过`Map`对象的`size`属性来了解该集合中当前有多少个键/值对。例如，对于上面的`Map`对象，我们可直接通过`imap.size`这个操作来获取`imap`对象中键/值对的个数。另外，`Map`对象自然还提供了其他一系列操作字典类数据结构的内置方法：

- `get`方法：该方法用于根据指定的“键”来获取对应的值。
- `set`方法：该方法用于根据指定的“键”来修改对应的值，如果指定的“键”不存在，就添加它。
- `has`方法：该方法用于判断指定的“键”是否存在于字典中。
- `delete`方法：该方法用于根据指定的“键”来删除字典中的键/值对。
- `clear`方法：该方法用于清空集合中所有的键/值对。

- keys方法：该方法会以一个遍历器的形式返回字典中所有键/值对的“键”。
- values方法：该方法会以一个遍历器的形式返回字典中所有键/值对的“值”。
- entires方法：该方法会以一个遍历器的形式返回字典中所有键/值对。
- forEach方法：该方法用于遍历字典，需要调用方指定一个回调函数的实参，用以执行循环操作。

WeakMap对象

顾名思义，这是一个弱化版的Map对象，它们也有与Set和WeakSet之间类似的不同之处。

- 首先，WeakMap对象的“键”只能存储对象类型的引用，不能存储基本类型的值。
- 其次，WeakMap对象中存储的“键”都是弱引用对象。这意味着，如果某个对象已经没有其他地方引用了，垃圾回收机制就会自动回收该对象所占用的内存，无论它是否还有引用存储在WeakMap对象中。

总而言之，WeakMap对象的大部分使用方式都与WeakSet对象基本相同，它也不支持values、keys、forEach等遍历方法，这里就不再复述了。

那么到底这两种弱化版本到底有什么用呢？基本上，如果我们想在数据结构中临时存储一些对象的引用，但又不想干扰垃圾回收机制对这些对象本身的管理，就可以使用WeakSet和WeakMap这类数据结构实现了。一个典型应用场景是，在网页的DOM上添加节点时，就可以采用WeakMap或WeakSet这类结构来存储它们。当相关DOM节点被移除时，其所对应的记录就会自动被移除。

A.4 综合练习

在初步学习了函数与对象的相关知识之后，我们打算在结束之前照例要综合演示一下在这篇笔记中介绍的知识点，顺便解决一下之前提出的问题，即如何将电话交换机及其测试程序修改成一个低耦合度的，相对可扩展的实现。到目前为止，我们已经将任务分离出来，封装成了下面这两个独立的函数。

```
function telephoneExchange(number) {
    switch (number) {
        case 1001:
            console.log('张三');
            break;
        case 1002:
            console.log('李四');
            break;
        case 1003:
            console.log('王五');
            break;
        case 1004:
            console.log('赵六');
            break;
        default:
            console.log('你拨打的是空号！');
            break;
    }
}

function testTelephoneExchange (callback) {
  for (let number = 1001; number < 1006; ++number) {
    callback(number);
```

```
    }
}
```

接下来，我们要解决switch语句带来的线路不可扩展问题。正如之前所说，这里应该用一个“电话簿”来管理线路的增加、删除、修改与查询。很明显，根据之前介绍的数据结构知识，这个“电话簿”应该属于字典类对象，所以，下面我们就用Map对象来实现一下这个“电话簿”。

```
class TelephoneExchange {
    constructor(names) {        // names 形参允许指定加入该电话交换机的初始名单
        this.mp = new Map();
        this.firstNum = 1001;     // 该电话交换机的第一个未占用的号码

        for(let name of names) {
            this.firstNum++;
            this.mp.set(this.firstNum, name); // 为初始名单分配电话号码
        }
    }

    add(name) {                           // 为新客户添加线路
        this.firstNum++;
        this.mp.set(this.firstNum, name);
    }

    delete(number) {                      // 删除线路
        this.mp.delete(number);
    }

    update (number, name) {                // 修改已有线路的所属人
        if (this.mp.has(number)) {
            this.mp.set(number, name);
        } else {
            console.log(number + '是空号！');
        }
    }

    call(number) {                        // 拨打指定线路
        if (this.mp.has(number)) {
            let name = this.mp.get(number);
            console.log('你拨打的用户是：  ' + name);
        } else {
            console.log(number + '是空号！');
        }
    }

    callAll() {                           // 拨打所有线路
        for (let number of this.mp.keys()) {
            this.call(number);
        }
    }
};
```

```
let phoneExch = new TelephoneExchange(['张三', '李四', '王五', '赵六']);
phoneExch.callAll();
console.log('-----------');
phoneExch.add('owlman');
phoneExch.callAll();
console.log('-----------');
phoneExch.delete(1002);
phoneExch.callAll();
console.log('-----------');
phoneExch.update(1003,'batman');
phoneExch.callAll();
console.log('-----------');
```

我们可以将其更新到之前创建的`TelephoneExchange/main.js`脚本文件中，然后在命令行终端中执行它，可以看到如下输出结果。

```
书稿\Examples\02_studynodejs on master [x!?] via v20.5.0
❯ node .\TelephoneExchange\main.js
你拨打的用户是： 张三
你拨打的用户是： 李四
你拨打的用户是： 王五
你拨打的用户是： 赵六
-----------
你拨打的用户是： 张三
你拨打的用户是： 李四
你拨打的用户是： 王五
你拨打的用户是： 赵六
你拨打的用户是： owlman
-----------
你拨打的用户是： 李四
你拨打的用户是： 王五
你拨打的用户是： 赵六
你拨打的用户是： owlman
-----------
你拨打的用户是： batman
你拨打的用户是： 王五
你拨打的用户是： 赵六
你拨打的用户是： owlman
-----------
```

图A-3 将程序封装为对象

如你所见，我们用`Map`对象创建了“电话簿”，然后围绕着它设置了添加、删除、修改以及呼叫线路的操作。现在我们的电话交换机在逻辑上更像一个可以应对实际需求的程序了。但是，它依然有着不小的问题，譬如，现在它只能根据电话号码查找用户，无法根据用户查找电话号码，而且一旦某条线路被删除，该线路的号码是无法被回收再利用的。另外，现在对电话交换机的测试也不只是将所有线路呼叫一遍这么简单了，我们需要重新编写可重用的测试任务函数，这会涉及到操作接口与操作实现的耦合度问题，这些都将是我们在下一篇附录中要介绍的内容了。